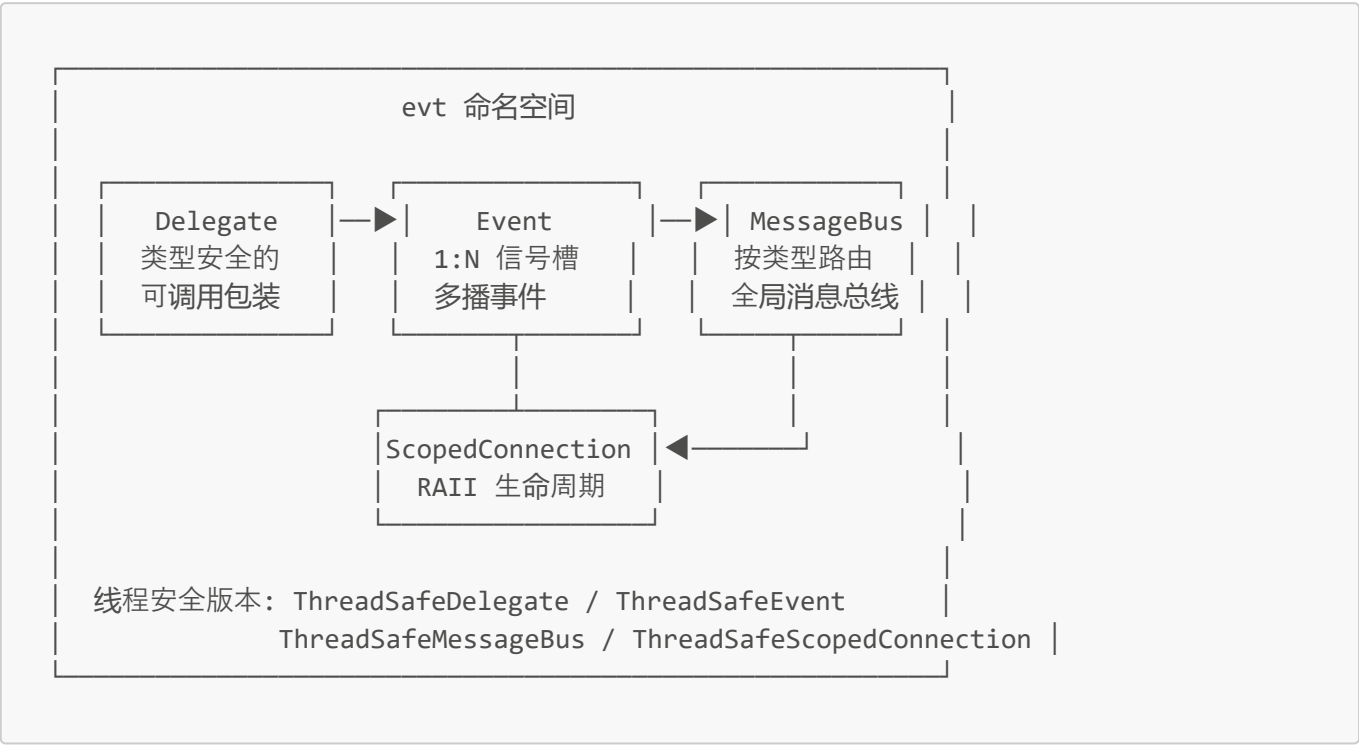


evt — 轻量级 C++ 事件/委托/消息总线框架

Header-only | C++ 14 | 零依赖（仅标准库） | 含线程安全版本

1. 架构总览



核心组件一览

组件	头文件	职责	线程安全
<code>Delegate<R(Args...)></code>	<code>eventBus.hpp</code>	类型安全的可调用对象包装（自由函数/Lambda/成员函数）	否
<code>Event<Args...></code>	<code>eventBus.hpp</code>	1:N 多播事件，支持订阅/取消/触发/延迟派发	否
<code>MessageBus</code>	<code>eventBus.hpp</code>	按消息类型自动路由的全局总线	否
<code>ScopedConnection</code>	<code>eventBus.hpp</code>	RAII 连接守卫，析构即断开	否
<code>ThreadSafe*</code> 系列	<code>eventBusThreadSafe.hpp</code>	上述组件的 mutex 保护版本	是

2. Delegate — 类型安全的可调用包装

`Delegate<R(Args...)>` 将自由函数、Lambda、成员函数统一为同一类型，支持有返回值的调用。

API

方法	说明
----	----

方法	说明
<code>Delegate::from(F&&)</code>	从 Lambda/可调用对象构造
<code>Delegate::from(R(*) (Args...))</code>	从自由函数指针构造
<code>Delegate::from(T*, R(T::*) (Args...))</code>	从对象 + 成员函数指针构造
<code>Delegate::from(const T*, R(T::*) (Args...) const)</code>	从 const 对象 + const 成员函数构造
<code>operator()(Args...)</code>	调用 (空 delegate 触发 assert)
<code>valid()</code>	是否持有有效可调用对象

示例

```
using namespace evt;

// 自由函数
auto d1 = Delegate<int(int, int)>::from(&add);
d1(3, 4); // => 7

// Lambda
auto d2 = Delegate<int(int, int)>::from([](int a, int b) { return a - b; });

// 成员函数
Calculator calc;
auto d3 = Delegate<int(int, int)>::from(&calc, &Calculator::multiply);

// const 成员函数
const Calculator cc;
auto d4 = Delegate<int(int, int)>::from(&cc, &Calculator::divide);
```

3. Event — 1:N 多播事件

`Event<Args...>` 是框架的核心。它维护一个 handler 列表，`emit()` 时依次调用所有订阅者。

API

方法	说明
<code>subscribe(handler)</code>	注册回调，返回 <code>ScopedConnection</code>
<code>emit(args...)</code>	同步触发所有订阅者
<code>operator()(args...)</code>	同 <code>emit()</code>
<code>post(args...)</code>	将参数入队，延迟触发
<code>flush()</code>	派发所有队列中的事件
<code>clear()</code>	清空所有订阅者

方法	说明
<code>size()</code>	当前订阅者数量
<code>pending_count()</code>	队列中待派发事件数
<code>clear_pending()</code>	清空队列（不触发）

3.1 基础：订阅 & 触发

```
using namespace evt;

Event<int, const std::string&> on_message;

// 订阅 — 返回 ScopedConnection, 析构自动断开
auto conn = on_message.subscribe([](int code, const std::string& msg) {
    std::cout << "code=" << code << ", msg=" << msg << "\n";
});

on_message.emit(200, "OK");           // 触发
on_message(404, "Not Found");         // operator() 等效
```

3.2 RAII 自动取消 & 手动断开

```
{
    auto conn = event.subscribe(handler);
    // conn 析构 → 自动 disconnect
}

// 或手动断开
conn.disconnect();
```

关键设计： `ScopedConnection` 不可拷贝，仅可移动。所有权唯一，杜绝悬空回调。

3.3 成员函数绑定

```
struct Player {
    std::string name;
    int hp = 100;
    void on_damage(int value) { hp -= value; }
};

Event<int> damage_event;
Player warrior{"战士"};
auto conn = damage_event.subscribe(
    Delegate<void(int)>::from(&warrior, &Player::on_damage)
);
damage_event.emit(20); // warrior.hp == 80
```

3.4 回调中取消订阅（快照安全）

`emit()` 内部先**快照** handler 列表再遍历，因此回调中 `disconnect()` 不会导致迭代器失效：

```
Event<> on_trigger;
ScopedConnection self_conn;

self_conn = on_trigger.subscribe([&]() {
    self_conn.disconnect(); // 安全：不影响当前快照遍历
});

on_trigger.emit(); // 正常执行
on_trigger.emit(); // 该回调不再触发
```

3.5 嵌套 emit

回调中可以再次 `emit()` 同一事件（递归），行为可预期（深度优先）：

```
Event<int> on_level;

auto conn = on_level.subscribe([&](int level) {
    if (level < 3) on_level.emit(level + 1); // 递归触发
});

on_level.emit(1); // 输出 1, 2, 3
```

3.6 延迟触发：post() + flush()

`post()` 将参数缓存到内部队列，`flush()` 时统一派发。适合**帧驱动**或**批量处理**场景：

```
Event<int, const std::string&> on_message;
auto conn = on_message.subscribe(handler);

on_message.post(200, "OK");
on_message.post(404, "Not Found");
// 此时 handler 不会被调用

on_message.flush(); // 统一触发 2 条
```

防递归：`flush()` 过程中新 `post()` 的事件不在本轮派发，可用 `while (pending_count() > 0) flush()` 实现广度优先逐层处理。

3.7 参数传递语义

Event 模板参数	handler 收到的	多订阅者安全	典型用途
Event<T>	每人一份拷贝	安全	通用场景（推荐）
Event<const T&>	共享同一引用	安全（只读）	避免拷贝大对象
Event<T&>	共享可变引用	需注意修改顺序	handler 需要修改原对象
Event<T&&>	右值引用	不安全（仅单订阅者）	所有权转移

3.8 Event 移动语义

Event 不可拷贝，但支持移动。移动后原 Event 变为空，已有的 ScopedConnection 仍然有效（通过 shared_ptr 跟踪）。

4. MessageBus — 按类型路由的消息总线

MessageBus 内部为每种消息类型维护一个独立的 Event<const Message&> 频道，发布时自动路由。

API

方法	说明
subscribe<Msg>(func)	订阅指定类型的消息
emit(msg)	立即发布消息，路由到对应频道
post(msg)	延迟发布，入队缓存
flush()	统一派发所有频道的待处理消息
clear<Msg>()	清空指定类型频道的订阅者
pending_count()	所有频道待处理消息总数

4.1 基础用法

```
// 定义消息类型（纯数据结构）
struct DamageEvent { int amount; std::string source; };
struct LoginEvent { std::string user; int login_time; };

evt::MessageBus bus;

// 订阅
auto c1 = bus.subscribe<DamageEvent>([](const DamageEvent& e) {
    std::cout << "damage=" << e.amount << "\n";
});
auto c2 = bus.subscribe<LoginEvent>([](const LoginEvent& e) {
    std::cout << "login=" << e.user << "\n";
});

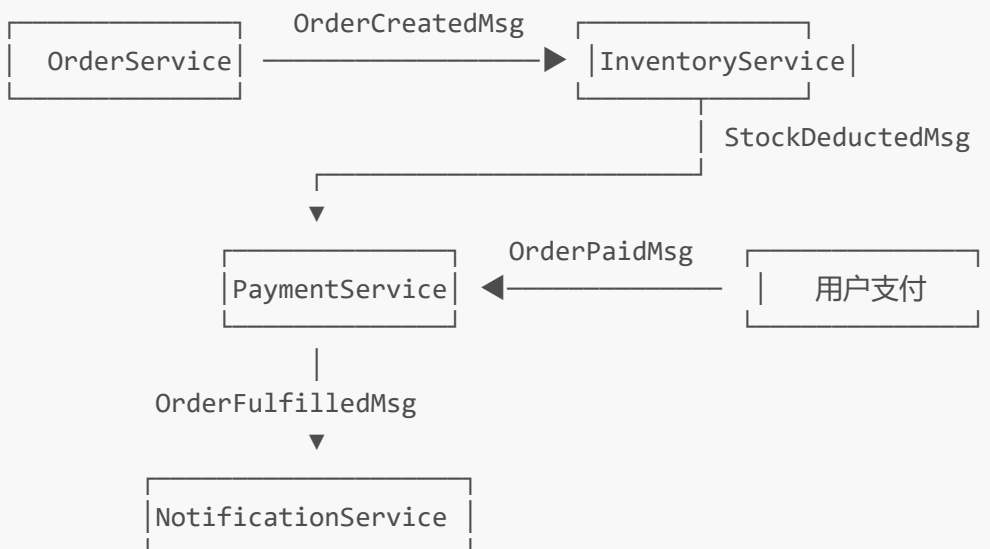
// 发布 — 自动路由到对应类型的订阅者
bus.emit(DamageEvent{30, "dragon"});
```

```
bus.emit(LoginEvent{"Alice", 1700000000});

// 无人订阅的类型静默跳过
bus.emit(LogoutEvent{"Bob"}); // 无输出, 不报错
```

4.2 业务架构模式

MessageBus 天然适合**服务间解耦**。每个服务只需知道消息契约（数据结构），不需要引用其他服务：



```
class InventoryService {
public:
    InventoryService(evt::MessageBus& bus) : bus_(bus) {
        conns_.push_back(
            bus_.subscribe<OrderCreatedMsg>([this](const OrderCreatedMsg& msg) {
                // 处理库存扣减...
                bus_.emit(StockDeductedMsg{msg.order_id, true, "OK"});
            })
        );
    }
private:
    evt::MessageBus& bus_;
    std::vector<evt::ScopedConnection> conns_; // RAII 管理所有订阅
};
```

服务析构时，`conns_` 自动断开所有订阅——**不会留下悬空回调**。

4.3 延迟发布

```
bus.post(DamageEvent{30, "dragon"});
bus.post(LoginEvent{"Alice", 0});
```

```
// 稍后统一派发
bus.flush();
```

5. ScopedConnection — RAII 连接管理

特性	说明
析构自动断开	离开作用域即取消订阅
不可拷贝	所有权唯一
可移动	通过 <code>std::move</code> 转移所有权
<code>disconnect()</code>	手动提前断开
<code>is_active()</code>	查询连接是否仍有效
Event 已销毁时安全	内部使用 <code>weak_ptr</code> , Event 先于 Connection 销毁不会崩溃

常见模式：在类中用 `std::vector<ScopedConnection>` 批量管理订阅生命周期。

6. 线程安全版本

头文件 `eventBusThreadSafe.hpp` 提供三个线程安全组件：

组件	对应非线性程安全版本	保护机制
<code>ThreadSafeScopedConnection</code>	<code>ScopedConnection</code>	<code>std::mutex</code> 保护所有状态访问
<code>ThreadSafeEvent<Args...></code>	<code>Event<Args...></code>	内部 <code>shared_ptr<State></code> + <code>mutex</code>
<code>ThreadSafeMessageBus</code>	<code>MessageBus</code>	频道表和各频道独立加锁

用法（与非线程安全版本 API 一致）

```
evt::ThreadSafeEvent<const Message&> event;

// 多线程订阅/触发均安全
auto conn = event.subscribe([](const Message& m) { /* ... */ });

std::thread t1([&] { event.emit(Message{1}); });
std::thread t2([&] { event.emit(Message{2}); });
t1.join(); t2.join();
```

设计要点

- `emit()` 先在锁内拷贝 handler 快照，再在**锁外**逐个调用——避免持锁调用用户代码导致死锁
- `ThreadSafeEvent` **不可移动**（移动语义与 `mutex` 冲突）
- `flush()` 先快照频道指针再逐个 `flush`，避免长时间持锁

7. 设计亮点

7.1 快照遍历

`emit()` / `flush()` 在调用 handler 前先复制 handler 列表:

```
auto snapshot = *m_handlers; // 值拷贝
for (const auto& slot : snapshot) {
    slot.handler(args...); // 回调中 subscribe/disconnect 不影响本轮遍历
}
```

这保证了回调中可以安全地增删订阅，不会崩溃或跳过。

7.2 异常透传

所有 `emit()` / `flush()` 均采用 **first-exception** 策略：即使某个 handler 抛异常，后续 handler 仍会执行，最后重新抛出第一个异常。

7.3 weak_ptr 防悬空

`subscribe()` 返回的 `ScopedConnection` 内部持有 `weak_ptr<HandlerList>`。如果 Event 先于 Connection 销毁，`disconnect()` 只是静默返回，不会访问已释放内存。

7.4 post/flush 防无限循环

`flush()` 先 `std::move` 出整个队列，再逐条派发。派发过程中新 `post()` 的事件进入新队列，不在本轮 flush 中处理。

8. 快速参考：选型指南

场景	推荐方案
简单回调/策略模式	<code>Delegate<R(Args...)></code>
观察者模式 / 1:N 通知	<code>Event<Args...></code>
跨模块解耦通信	<code>MessageBus</code>
帧驱动/批量处理	<code>post()</code> + <code>flush()</code>
多线程环境	<code>ThreadSafeEvent</code> / <code>ThreadSafeMessageBus</code>
所有权转移（单订阅者）	<code>Event<T&&></code>
避免拷贝大对象	<code>Event<const T&></code> 或 <code>MessageBus</code> （默认 <code>const&</code> ）

9. 最小完整示例


```
#include "generic/eventBus.hpp"
#include <iostream>
#include <string>

struct ChatMessage {
    std::string user;
    std::string text;
};

int main() {
    evt::MessageBus bus;

    // 服务 A: 日志
    auto c1 = bus.subscribe<ChatMessage>([](const ChatMessage& m) {
        std::cout << "[LOG] " << m.user << ": " << m.text << "\n";
    });

    // 服务 B: 过滤敏感词
    auto c2 = bus.subscribe<ChatMessage>([](const ChatMessage& m) {
        if (m.text.find("bad") != std::string::npos)
            std::cout << "[WARN] 敏感词检测: " << m.user << "\n";
    });

    bus.emit(ChatMessage{"Alice", "hello world"});
    bus.emit(ChatMessage{"Bob", "this is bad"});

    c1.disconnect(); // 日志服务下线
    bus.emit(ChatMessage{"Eve", "bad again"});
    // 只有过滤服务收到

    return 0;
}
```

输出:

```
[LOG] Alice: hello world
[LOG] Bob: this is bad
[WARN] 敏感词检测: Bob
[WARN] 敏感词检测: Eve
```